

Étape 2 : Debug Dynamique & Extraction de Clé

Objectif : Utiliser GDB pour extraire la clé de chiffrement du ransomware en mémoire.

Objectifs de l'Étape

À la fin de cette étape, vous devez :

- ✓ Lancer le ransomware dans GDB
 - ✓ Placer des breakpoints stratégiques
 - ✓ Inspecter la mémoire pour trouver la clé
 - ✓ Extraire la clé de chiffrement complète
 - ✓ Documenter la méthode utilisée
-

Fichiers à Utiliser

Vous allez utiliser le binaire compilé à l'Étape 1 :

```
../Etape1_AnalyseStatique/ransomware
```

Instructions

1. Préparation

1.1 Créer un Environnement de Test Sécurisé

```
cd Etape2_DebugDynamique/  
  
# Créer des fichiers de test  
mkdir -p test_files  
echo "Document de test pour debug" > test_files/test.txt  
echo "Autre fichier de test" > test_files/data.txt
```

1.2 Vérifier que GDB est Installé

```
gdb --version
```

Si pas installé :

```
sudo apt-get install gdb
```

2. Lancement de GDB

2.1 Démarrer GDB avec le Ransomware

```
# Copier le binaire dans ce dossier
cp ../Etape1_AnalyseStatique/ransomware .

# Lancer GDB
gdb ./ransomware
```

3. Placement des Breakpoints

3.1 Identifier les Fonctions Intéressantes

Dans GDB, listez les fonctions :

```
(gdb) info functions
```



Question 1 : Combien de fonctions voyez-vous ?

3.2 Placer un Breakpoint sur `main`

```
(gdb) break main
```

Vous devriez voir :

```
Breakpoint 1 at 0x... : file ransomware.c, line 195.
```



Question 2 : À quelle adresse mémoire se trouve `main` ?

3.3 Placer un Breakpoint sur `encrypt_file`

Cette fonction est appelée pour chaque fichier à chiffrer. C'est là que la clé est utilisée !

3.4 Placer un Breakpoint sur **encrypt**

C'est la fonction qui fait réellement le chiffrement.

Vérification :

```
(gdb) info breakpoints
```

Vous devriez voir 3 breakpoints listés.

4. Exécution et Extraction de la Clé

4.1 Lancer le Programme

```
(gdb) run ./test_files
```

Le programme s'arrête au premier breakpoint (main).

```
Breakpoint 1, main (argc=2, argv=0x7fffffffef3b8) at ransomware.c:195
195      int main(int argc, char *argv[]) {
```

4.2 Afficher les Variables Globales

La clé est définie comme variable globale : **encryption_key** Variable globale = Variable accessible depuis n'importe quelle fonction.

Printez la clé dans GDB :

```
**📝 Question 3 :** Que voyez-vous ?
```

Si vous voyez un nombre (adresse), essayez :

```
```gdb
(gdb) print (char*)encryption_key
```

Ou mieux :

```
(gdb) x/s encryption_key
```

Cette commande signifie : "Examine as String"

 **Question 4** : Quelle est la clé de chiffrement complète ?

---

### 4.3 Examiner la Clé en Hexadécimal

```
(gdb) x/20xb encryption_key
```

Signification :

- `x` = examine
- `/20xb` = 20 bytes en hexadécimal

 **Question 5** : Notez les 20 premiers bytes en hexa.

---

### 4.4 Vérifier la Longueur de la Clé

```
(gdb) print KEY_LENGTH
```

 **Question 6** : Quelle est la longueur de la clé ?

---

## 5. Analyse Approfondie

### 5.1 Continuer jusqu'à `encrypt_file`

```
(gdb) continue
```

Le programme s'arrête à `encrypt_file` quand il trouve un fichier à chiffrer.

```
(gdb) info args
```

Cela affiche les arguments de la fonction.

 **Question 7** : Quel fichier est en train d'être chiffré ?

---

### 5.2 Analyser le Breakpoint dans `encrypt`

```
(gdb) continue
```

Le programme s'arrête dans `encrypt`.

```
(gdb) info args
```

 **Question 8** : Quels sont les 4 arguments de `encrypt` ?

Astuce : Regardez la signature dans le code :

```
void encrypt(unsigned char* data, size_t data_len,
 unsigned char* key, size_t key_len)
```

---

### 5.3 Examiner les Données Avant Chiffrement

```
(gdb) x/50c data
```

Cela affiche les 50 premiers caractères des données.

 **Question 9** : Voyez-vous le contenu du fichier original ?

---

### 5.4 Avancer d'une Instruction

```
(gdb) next
```

Ou pour entrer dans les fonctions :

```
(gdb) step
```

---

## 6. Extraction et Vérification

### 6.1 Sauvegarder la Clé

Créez un fichier `key_extracted.txt` :

```
echo "<key>" > key_extracted.txt
```

(Remplacez par la vraie clé extraite avec GDB)

---

### 6.2 Vérifier la Clé en Hexadécimal

```
xxd key_extracted.txt
```

Comparez avec ce que vous avez vu dans GDB (`x/20xb encryption_key`).

## 7. Commandes GDB Utiles

Commande	Description
<code>run [args]</code>	Lancer le programme
<code>break fonction</code>	Placer un breakpoint
<code>break fichier.c:ligne</code>	Breakpoint sur une ligne
<code>info breakpoints</code>	Lister les breakpoints
<code>delete N</code>	Supprimer le breakpoint N
<code>continue</code>	Continuer l'exécution
<code>next</code>	Exécuter la ligne suivante
<code>step</code>	Entrer dans la fonction
<code>print variable</code>	Afficher une variable
<code>x/Nxb adresse</code>	Examiner N bytes en hexa
<code>x/s adresse</code>	Examiner comme string
<code>info locals</code>	Variables locales
<code>info args</code>	Arguments de fonction
<code>backtrace</code>	Stack trace
<code>quit</code>	Quitter GDB



## Livrables de l'Étape 2

À remettre :

- ✓ Fichier `key_extracted.txt` avec la clé complète
- ✓ Fichier `gdb_commands.txt` avec les commandes GDB utilisées
- ✓ Capture d'écran de GDB montrant la clé (optionnel mais recommandé)
- ✓ Réponses aux 9 questions



## Prochaine Étape

Avec la clé extraite, passez à **l'Étape 3 : Analyse du Serveur C2** pour comprendre comment le ransomware communique !

---

**Bon debug !** 🐛🔍